DOM Selectors & Manipulation

JavaScript Document Object Model Quick Reference Guide

1. Selecting Elements

Retrieving elements from the DOM is the first step in manipulation. JavaScript provides several methods depending on whether you need a single element or a collection.

- `document.getElementById('id_name')`: Selects a single element based on its unique ID. Returns the element object.
  Example: `const title = document.getElementById('title');`
- `document.getElementsByClassName('class_name')`: Selects all elements possessing a specific class. Returns an **HTMLCollection**.
- `document.querySelector('selector')`: Selects elements using standard CSS syntax (e.g., `#id`, `.class`, `tag`). Returns only the **first** matching element.
- `document.querySelectorAll('selector')`: Selects all elements matching the CSS selector. Returns a **NodeList**.

2. Accessing & Modifying Attributes

Once an element is selected, you can read or overwrite its HTML attributes (like `src`, `href`, `class`, or `id`).

- `getAttribute('attribute_name')`: Retrieves the current value of an attribute.
  Example: `title.getAttribute('class');`
- `setAttribute('attribute_name', 'value')`: Sets a new value for an attribute.

> ⚠️ **Important Note on setAttribute**
>
> This method strictly **overwrites** existing values. To retain older classes while adding new ones, you must pass both into the string:
> `element.setAttribute('class', 'old-class new-class')`

3. Extracting Content

These three properties extract content from an HTML element but behave differently based on visibility and nested HTML tags.

Property	Behavior	Best Use Case
`innerText`	Returns only the text that is physically visible on the screen. It actively ignores text hidden via CSS (e.g., `display: none`).	When you need the text exactly as the user currently sees it.

4. Manipulating Styles

To manipulate CSS properties via JavaScript, access the `.style` object on the selected element. The most important rule to remember is that CSS properties using hyphens (like `background-color`) are converted to **camelCase** in JavaScript (`backgroundColor`).

```
const title = document.querySelector('#title'); // Modifying styles directly
title.style.backgroundColor = 'green'; title.style.padding = '15px';
title.style.borderRadius = '5px'; title.style.color = '#ffffff';
```

5. NodeList vs HTMLCollection (Crucial Difference)

When selecting multiple elements, the DOM returns specific collections, not standard JavaScript arrays. Understanding how to iterate over them is critical to avoiding errors.

NodeList

Returned by: `querySelectorAll()`

Characteristics: Contains a built-in `forEach()` method, making it easy to iterate over, though it lacks pure array methods like `map()`.

```
const allH2 = document.querySelectorAll('h2');
// Directly iterate using forEach
allH2.forEach(function(h2Element) { h2Element.style.color = 'red';
});
```

Note: You cannot style a NodeList directly (`allH2.style.color = 'red'` will fail). You must use an index (`allH2[0]`) or loop over them.

HTMLCollection

Returned by: `getElementsByClassName()`

Characteristics: Does **not** contain a `forEach()` method natively. Attempting to loop over it directly will result in a runtime error.

```
const classList = document.getElementsByClassName('item');
// 1. Convert to Array first const convertedArray = Array.from(classList);
// 2. Loop over the newly created array
convertedArray.forEach(function(item) { item.style.color = 'orange'; });
```

Fix: Always convert an HTMLCollection into a standard Array first using `Array.from()` before iterating.

6. Advanced querySelector Techniques

The `querySelector` method supports complex CSS selectors, including pseudo-classes. This allows you to specifically target list items or nested elements without needing additional loops.

```
// 1. Selecting the first list item const firstItem = document.querySelector('ul li:first-child'); firstItem.style.color = 'green'; // 2. Selecting specific nth-child const secondItem = document.querySelector('ul li:nth-child(2)'); secondItem.style.backgroundColor = 'yellow'; secondItem.innerText = 'Updated Item Text'; // 3. Selecting elements by attribute specifically const passInput = document.querySelector('input[type="password"]');
```

7. Extracting Content: Code in Action

To clearly see the difference between `innerText`, `textContent`, and `innerHTML`, consider the following HTML structure where a span of text is hidden via CSS.

```
<!-- HTML Structure --> <h1 id="title">DOM Learning <span style="display: none;">Hidden Text</span></h1> <script> const title = document.getElementById('title'); // 1. innerText - strictly reads what is visible on screen console.log(title.innerText); // Output: "DOM Learning" // 2. textContent - retrieves all raw text nodes, bypassing CSS visibility console.log(title.textContent); // Output: "DOM Learning Hidden Text" // 3. innerHTML - retrieves the full text along with raw HTML tags console.log(title.innerHTML); // Output: "DOM Learning <span style="display: none;">Hidden Text</span>" </script>
```